

Contents

Guide 1 — Web Scraping (simple, and how to defend it)	1
1. The idea, in three sentences	1
2. The tools, and the one-line reason for each	1
3. How it works — six steps	1
4. One worked example (a Djezzy plan)	1
5. The files (one line each — in src/lib/scrapper/)	2
6. Jury questions — ready answers	2

Guide 1 — Web Scraping (simple, and how to defend it)

A short, plain guide to the web-scraping part of Mobile Match Algeria: what it does, the tools, how it works, one worked example, and ready answers for the jury.

1. The idea, in three sentences

The three operators publish their plans on their websites, but none of them offers an **API** (a clean data feed a program can ask). So our program acts like a visitor: it **opens the public pages, reads the HTML, and pulls out the numbers** (price, data, minutes, SMS, validity) into our database. That is web scraping.

2. The tools, and the one-line reason for each

Tool	What it does	Why we need it
Playwright	drives a real web browser from code	plain requests are blocked by the operators' anti-bot; a real browser gets through
Microsoft Edge (headless)	the real browser Playwright drives, with no window shown	a genuine installed browser is the hardest to detect
Cheerio	reads the downloaded HTML and picks out values with CSS selectors	light and fast once we have the HTML
node-cron	runs the scrape once a day, by itself	no extra service to host

In one line: *Playwright + headless Edge fetch the page, Cheerio reads it, node-cron runs it daily.*

3. How it works — six steps

1. **Fetch** — open the operator page in headless Edge.
2. **Parse** — Cheerio reads the HTML and pulls out price / data / minutes / SMS / validity.
3. **Normalise** — turn the text into clean numbers ("200 GO" → 200, "Pour 4000 DA" → 4000).
4. **Validate** — every offer must pass 12 sanity rules before it can be saved.
5. **Save** — new plan → insert; changed price → record it and alert users; plan gone from the site → mark inactive.
6. **Finish** — rebuild the search index and send notifications.

A full run for all three operators takes a few seconds.

4. One worked example (a Djezzy plan)

The page gives us a card like this (simplified):

```
<article class="price-card">
  <li><i class="icon-Internet"></i>      200 GO Internet</li>
  <li><i class="icon-Appels-Illimite"></i> APPELS ILLIMITÉS VERS TOUS LES RÉSEAUX</li>
  <li><i class="icon-SMS-illimite"></i>    SMS ILLIMITÉS VERS DJEZZY</li>
  <li><i class="icon-Courrier"></i>       30 SMS VERS LES AUTRES RÉSEAUX</li>
  <li><i class="icon-Historique"></i>    30 Jours Validité</li>
  <li>                                  Pour 4000 DA</li>
</article>
```

The parser reads each line **by its icon class**, the normalisers turn the text into numbers, and we get one clean record:

```
{ "name": "Djezzy LEGEND 4000", "priceDA": 4000, "dataGB": 200,
  "voiceMinutes": -1, "smsCount": -1, "validityDays": 30, // -1 means unlimited
  "features": ["Unlimited calls (all networks)", "Unlimited Djezzy SMS",
    "30 SMS to other networks"] }
```

On the card the user then sees: **200 GB · Unlimited (all networks) · Unlimited (Djezzy) + 30 off-net · 4000 DA.**

One credit detail worth knowing. Some plans show a recharge credit like “3 000 DA CRÉDIT”. Djezzy’s page misspells *crédit* as *crédit* (wrong accent), so our finder uses the pattern `/cr[éèe]dit/i`, which accepts **é, è, or e** — so the green credit chip still appears. (Lesson: real operator HTML has typos; our code has to be forgiving.)

5. The files (one line each — in **src/lib/scrapper/**)

- **fetch.ts** — opens each page in headless Edge; retries a few times, and if a site stays down a verified fallback takes over.
 - **validate.ts** — the French / English / Arabic text-to-number parsers, plus the 12 rules every offer must pass.
 - **djezzy.ts / ooredoo.ts / mobilis.ts** — one file per operator; each knows its own site’s HTML layout.
 - **index.ts** — runs all three, saves to the database, deactivates offers that vanished, sends alerts.
 - **instrumentation.ts** — starts the daily 03:00 job automatically when the server boots.
-

6. Jury questions — ready answers

Q — Why scraping and not an API? None of the three Algerian operators offers a public plan API. The literature says scraping is the standard, accepted method when data is published only as HTML. It wasn’t a shortcut — it was the only option.

Q — Is scraping legal and ethical? We read only public marketing pages (the same ones any visitor sees), we respect the site's `robots.txt` and add delays between requests, we identify our browser, and we store no personal data.

Q — Why run a whole browser? Isn't that heavy? The operators block plain requests and build their pages with JavaScript, so a simple request returns nothing useful. A real browser is the reliable way through. The extra memory doesn't matter because we run a few times a day, not in a tight loop.

Q — What if an operator changes its website? That operator's scraper fails safely and a verified fallback dataset takes over, so the app never goes blank. We also keep saved copies of the pages as tests, which catch a layout change quickly.

Q — How do you stop bad data getting in? Two safeguards: the parsers are "unit-anchored" (a bare "200" with no unit returns 0, so a price can't be misread as data), and every offer must pass 12 validation rules before it is saved.

Q — How often does it run, and who starts it? Automatically every day at 03:00 Algiers time, plus once on startup if the data is missing or old, plus a manual button in the admin panel.

Q — How do you handle the three languages? The parsers recognise French, English, and Arabic units (for data: Go, GB, and the Arabic word for "giga"; for validity: jours, mois, and the Arabic words for day and month), so the same code reads all three operators.

Q — Some prices are inside images. How do you handle those? A few families (Mobilis Revolution, Djazzy iZZY) publish prices as images, which can't be read as text. Those are entered by hand and re-checked regularly. We tested OCR but it was only 29–43 % accurate, so we rejected it; a paid vision API is the future option (in the Perspectives chapter).

Q — How do you detect new offers and price changes? On each run we compare to the database: a new plan is inserted, a changed price is logged to the price history (and users are alerted on a drop), and a plan that disappeared is marked inactive — with a guard so a broken run can't wipe the catalogue.

Q — What is the main weakness or risk? A site could block us or redesign its pages. We reduce that with a real browser, spaced retries, a circuit-breaker, and the fallback dataset — and we discuss it honestly in the Perspectives chapter.

Next guide: the recommendation engine (the formula and how to defend it). Tell me when you want it.